

Report

Hyperparamater Tuning

These are the configurations and results of every explored architecture on each dataset for the project.

MNIST

Shallow MLP

LR	Batch	Opt	Dropout	Acc	Runtime
0.001	32	adam	0.2	0.9786	93.903811s
0.001	64	adam	0.5	0.9699	50.474093s
0.001	32	adam	0.5	0.968	92.709780s
0.001	128	adam	0.5	0.9648	31.938828s
0.01	32	sgd	0.5	0.955	95.026807s
0.0001	32	adam	0.5	0.9529	94.956273s
0.01	32	adam	0.5	0.9325	96.934077s
0.001	32	sgd	0.2	0.882	96.404471s
0.001	32	sgd	0.5	0.8659	95.066064s
0.001	64	sgd	0.5	0.7856	50.235637s
0.001	128	sgd	0.5	0.5202	31.320126s
0.0001	32	sgd	0.5	0.3415	94.331081s

Medium MLP

LR	Batch	Opt	Dropout	Acc	Runtime
0.001	32	adam	0.2	0.9786	93.903811s
0.001	64	adam	0.5	0.9699	50.474093s
0.001	32	adam	0.5	0.968	92.709780s
0.001	128	adam	0.5	0.9648	31.938828s
0.01	32	sgd	0.5	0.955	95.026807s
0.0001	32	adam	0.5	0.9529	94.956273s
0.01	32	adam	0.5	0.9325	96.934077s
0.001	32	sgd	0.2	0.882	96.404471s
0.001	32	sgd	0.5	0.8659	95.066064s
0.001	64	sgd	0.5	0.7856	50.235637s
0.001	128	sgd	0.5	0.5202	31.320126s
0.0001	32	sgd	0.5	0.3415	94.331081s

Deep MLP

LR	Batch	Opt	Dropout	Acc	Runtime
0.001	32	adam	0.2	0.979	155.028317s
0.001	64	adam	0.5	0.974	83.823792s
0.001	128	adam	0.5	0.9728	48.206226s
0.001	32	adam	0.5	0.9692	158.598686s
0.0001	32	adam	0.5	0.9608	152.521758s
0.01	32	sgd	0.5	0.9032	141.336961s
0.001	32	sgd	0.5	0.1149	142.345350s
0.001	64	sgd	0.5	0.1106	79.070498s
0.001	32	sgd	0.2	0.1091	145.432496s
0.001	128	sgd	0.5	0.0973	44.535114s
0.0001	32	sgd	0.5	0.0963	143.586129s
0.01	32	adam	0.5	0.0952	156.156998s

Simple CNN

LR	Batch	L2	Acc	Runtime
0.001	64	0.0	0.9914	65.917064s
0.001	64	0.005	0.9897	61.270601s
0.001	64	0.0001	0.9885	64.817863s
0.001	32	0.0001	0.9882	273.198989s
0.0001	32	0.0001	0.9882	117.321126s
0.001	128	0.0001	0.987	37.414027s
0.01	32	0.0001	0.9866	116.705469s

Enhanced CNN

LR	Batch	L2	Acc	Runtime
0.001	64	0.0001	0.9951	74.394916s
0.001	64	0.0	0.9936	70.366497s
0.001	128	0.0001	0.992	53.862178s
0.01	32	0.0001	0.9904	138.284246s
0.001	64	0.005	0.9902	70.026856s
0.0001	32	0.0001	0.9898	139.986008s
0.001	32	0.0001	0.9883	223.882436s

CIFAR

Shallow MLP

+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
LR	Batch	Opt	Dropout	Acc	Runtime	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
0.01	32	sgd	0.5	0.476	84.069314s	
0.0001	32	adam	0.5	0.4726	87.730901s	
0.001	32	adam	0.2	0.4488	87.590577s	
0.001	128	adam	0.5	0.4154	33.142347s	
0.001	64	adam	0.5	0.3742	52.177993s	
0.001	32	sgd	0.2	0.3618	84.097001s	
0.001	32	sgd	0.5	0.3568	85.430178s	
0.001	32	adam	0.5	0.3474	86.276390s	
0.001	64	sgd	0.5	0.3118	48.823452s	
0.001	128	sgd	0.5	0.2798	32.618860s	
0.0001	32	sgd	0.5	0.229	84.205199s	
0.01	32	adam	0.5	0.103	88.727425s	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

Medium MLP

+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
LR	Batch	Opt	Dropout	Acc	Runtime	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
0.0001	32	adam	0.5	0.4408	121.820131s	
0.001	32	adam	0.2	0.4266	123.817272s	
0.01	32	sgd	0.5	0.4104	113.511294s	
0.001	64	adam	0.5	0.2828	64.022769s	
0.001	128	adam	0.5	0.2798	38.705480s	
0.001	32	sgd	0.2	0.243	116.079208s	
0.001	32	adam	0.5	0.2376	123.679928s	
0.001	32	sgd	0.5	0.2018	116.933495s	
0.001	64	sgd	0.5	0.1754	66.425610s	
0.001	128	sgd	0.5	0.1484	38.894269s	
0.0001	32	sgd	0.5	0.1422	113.415617s	
0.01	32	adam	0.5	0.0996	119.850072s	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+

Deep MLP

+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
LR	Batch	Opt	Dropout	Acc	Runtime	
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
0.001	32	adam	0.2	0.37	147.654079s	
0.0001	32	adam	0.5	0.3534	145.696236s	

	0.01		32		sgd		0.5		0.194		135.418625s	
	0.001		32		adam		0.5		0.1738		150.291151s	
	0.001		128		adam		0.5		0.1686		44.880285s	
	0.001		64		adam		0.5		0.1664		83.366021s	
	0.001		32		sgd		0.2		0.1616		138.118310s	
	0.001		128		sgd		0.5		0.1054		47.246877s	
	0.01		32		adam		0.5		0.104		148.897285s	
	0.0001		32		sgd		0.5		0.1024		135.186240s	
	0.001		32		sgd		0.5		0.1		133.703073s	
	0.001		64		sgd		0.5		0.0974		78.475411s	
+-----+-----+-----+-----+-----+-----+												

Simple CNN

+-----+-----+-----+-----+-----+						
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
+-----+-----+-----+-----+-----+						

Enhanced CNN

+-----+-----+-----+-----+-----+						
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
	LR		Batch		L2	
+-----+-----+-----+-----+-----+						

Test Results

MNIST

MLP

Arch	LR	Batch	Opt	Dropout	Acc	Runtime
deep_mlp	0.001	64	adam	0.2	0.9823	30.268441s
medium_mlp	0.001	64	adam	0.2	0.9796	26.385989s
shallow_mlp	0.001	64	adam	0.2	0.9782	22.341757s

On the mnist dataset, the best configurations for the different depth mlp architectures were stable:

- learning_rate = .001. Neither of the runs with lr=.0001 or lr=.01, were able to score higher than my baseline run which got a .968, but lr=.01 was able to do the best out of all of the SGD runs and outperformed both the not .0001 adam runs, which is interesting.
- batch_size = 64. I used 32 as the default batch_size, but 64 seems to receive comprable or better performance in most cases while decreasing the runtime by half. I regret the choice to use 32 as the default batch_size.
- optimizer = adam. All of the adam runs were able to acheive reasonable peformance on the MNIST set. The lowest adam scored an accuracy of ~.93. The SGD runs were much more sensitive to the other hyperparamaters and scored all over the place, highest = .95, lowest = .35.
- dropout = .2. It seems that dropout of .5 seemed to underfit, performing slightly worse than .2 with both adam and SGD.

CNN

Arch	LR	Batch	L2	Acc	Runtime
simple_conv	0.001	64	0.0	0.9918	58.170087s
enhanced_conv	0.001	64	0.0001	0.9915	78.339576s

I found similar values for the two shared parameters with mlp architectures. Medium learning rate, medium batch size. The regularization constant did differ between architecture complexities for the CNNs. I think this can be accounted for by their differing expressive capability. The enhanced cnn will be more likely to over-fit, because of its deeper architecture, and the simple cnn more general because of its reduced complexity. L2 regularization would help smooth out a model with higher variance, but potentially increase the bias of a more or adequately biased model. The simple cnn did perform better on test, so I think it was adequately biased as is, but the difference in accuracy is vanishingly small.

CIFAR10

MLP

Arch	LR	Batch	Opt	Dropout	Acc	Runtime
medium_mlp	0.0001	32	adam	0.2	0.5264	55.688885s
shallow_mlp	0.01	32	sgd	0.2	0.5031	44.092095s
deep_mlp	0.001	32	adam	0.2	0.3829	97.297526s

Interesting differences in parameter configurations across architectures for mlps on CIFAR 10 with albeit pretty poor accuracy. Best dropout was .2 for all following the best configuration for mnist. It is uprisng that you can achieve solid performance even in a shallow network even when half the weights are dropped. Best batch_size for all three architectures was 32, which is not what I had expected given the mnist data. My best guess for what is happening is that the model is not able to learn important features, shown by the low accuracy. The small batch sizes create noisier more general gradient updates, that are better able to generalize compared to configurations with larger batches. Which might be learning less general features from the training data. The learning rates were all over the place. Interestingly lr=.01 again achieved relatively high performance when paired with the SGD optimizer, similar behavior to the MNIST configurations.

CNN

Arch	LR	Batch	L2	Acc	Runtime
enhanced_conv	0.001	64	0.0	0.746	68.194038s
simple_conv	0.001	32	0.0	0.6781	104.134194s

Much worse performance all around on the CIFAR10 set compared to MNIST results. CNNs did however manage much better performance than MLPs on the set.

- lr = .001. Again following the trend from all other architectures in the project.
- l2 = 0.0. No weight decay tracks given the low accuracy of the models. It's not over-fitting, or doing a good job learning really probably from insufficient model complexity.
- batch-size = 64 for enhanced, 32 for simple. It does seem like once the model complexity increases to some threshold, it begins to prefer larger batch sizes. Lower batch sizes are able to perform better in validation when the model accuracy is limited by it's complexity, but as the complexity increases and the potential for greater accuracy, larger batches begin to work better? I would need to do more testing, but it's an interesting thought.

Key Challenges

A key challenge I faced while implementing this project was data aggregation. The requirements state that for each data set, every of 5 architectures, should be evaluated on at least 10 hyperparamater sets, to find the best model configuration for the data set and given architecture. That's $2 \times 5 \times 10 =$ a minimum of 100

different training runs whose details need to be reported out. This is almost impossible to do manually. Instead I have each training run log a json object, to an append only log in `./run_log.jsonl`. This log acts as both a data store, from which we can create tables with `./report.py`, and a cache, so that the training harness, knows which configurations have been tried before and can be skipped. Reporting can then easily be defined as a query on the structured `run_log` output to generate the tables you see in this report.